

[6강] Advanced RAG (1)

조현수

이화여자대학교 인공지능학과 조교수

저작권 안내

(주)업스테이지가 제공하는 모든 교육 콘텐츠의 지식재산권은
운영 주체인 (주)업스테이지 또는 해당 저작물의 적법한 관리자에게 귀속되어 있습니다.

콘텐츠 일부 또는 전부를 복사, 복제, 판매, 재판매 공개, 공유 등을 할 수 없습니다.

유출될 경우 지식재산권 침해에 대한 책임을 부담할 수 있습니다.

유출에 해당하여 금지되는 행위의 예시는 다음과 같습니다.

- 콘텐츠를 재가공하여 온/오프라인으로 공개하는 행위
- 콘텐츠의 일부 또는 전부를 이용하여 인쇄물을 만드는 행위
- 콘텐츠의 전부 또는 일부를 녹취 또는 녹화하거나 녹취록을 작성하는 행위
- 콘텐츠의 전부 또는 일부를 스크린 캡처하거나 카메라로 촬영하는 행위
- 지인을 포함한 제3자에게 콘텐츠의 일부 또는 전부를 공유하는 행위
- 다른 정보와 결합하여 Upstage Education의 콘텐츠임을 알아볼 수 있는 저작물을 작성, 공개하는 행위
- 제공된 데이터의 일부 혹은 전부를 Upstage Education 프로젝트/실습 수행 이외의 목적으로 사용하는 행위

강의 목표

학습 목표

- RAG의 각 구성 요소를 강화하는 고도화 기법의 필요성을 이해하고 각 기법에 대해 설명할 수 있다.
- 전체 RAG 파이프라인 고도화 기법들의 배경을 이해하고 각 기법에 대해 설명할 수 있다.

01. Advanced RAG 기법

(1) Query 강화

(2) Chunking 기법

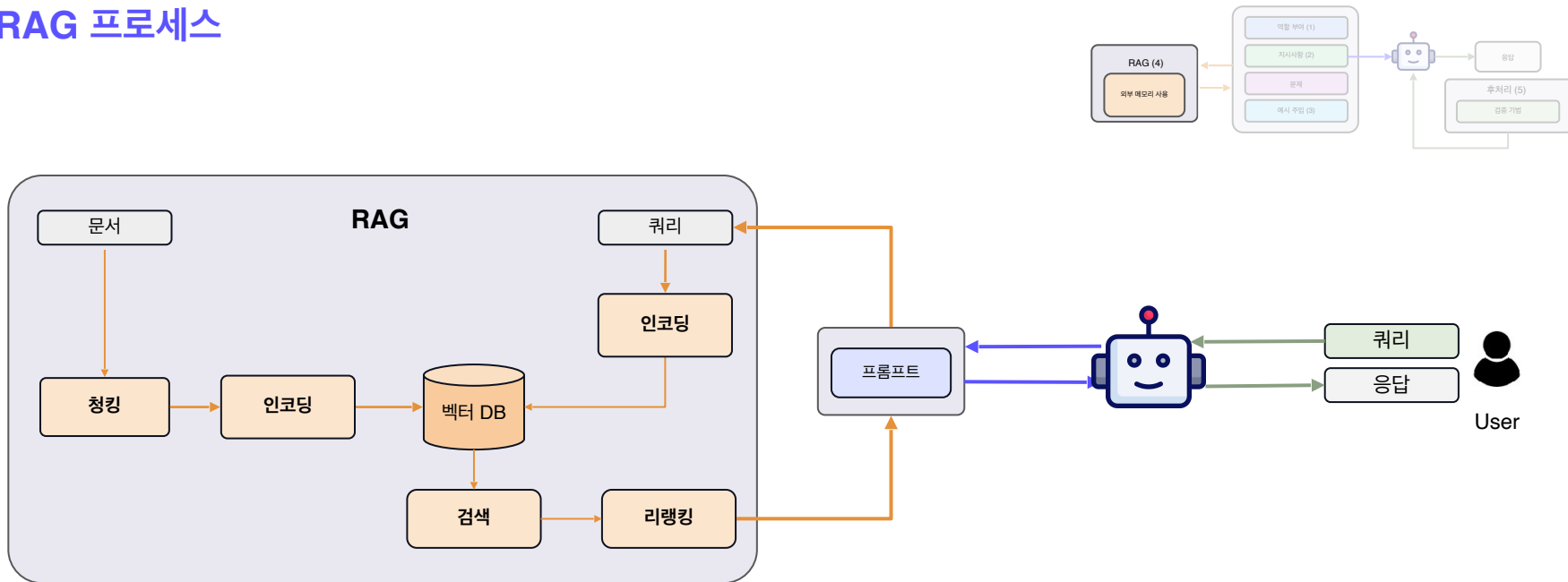
(3) Reranking 기법

Advanced RAG 기법

RAG의 각 구성요소에 대한 고도화 기법

Recap

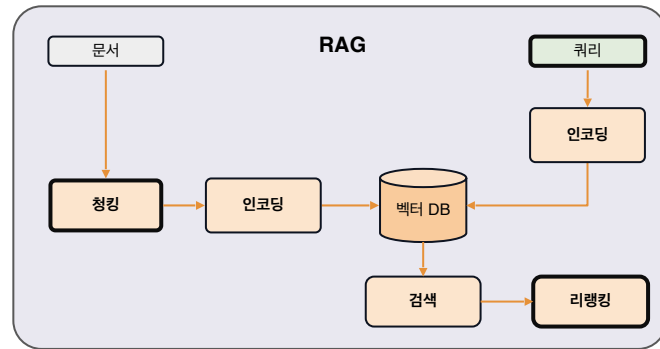
RAG 프로세스



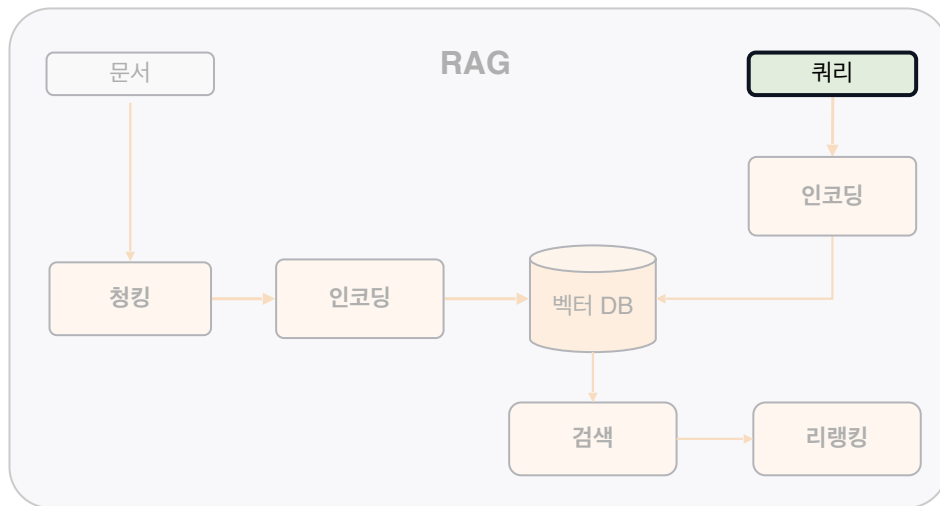
RAG 고도화 기법

Overview

1. **Query 강화 고도화 기법** (e.g. HyDE, RAGRouter)
2. **Chunking 전략** (e.g. Small2Big)
3. **Reranking 기법** (e.g. ListConRanker)



1. Query 강화 고도화 기법



RAG의 문제 상황

- 문제상황

- 사용자 질문이 짧고 추상적일 경우?
- 문서가 쓰는 용어와 사용자 표현이 다른경우?

- Query Transformation이란

- RAG 검색 단계 전에 사용자의 원 질문을 검색에 유리한 형태로 바꾸는 전처리 기법
- 목표는 같은 의미를 더 구체적이고 다양한 표현으로 재구성해서 검색 적합도를 높이는 것

- How?

- 재작성 질문을 더 명확하게 다시 씀
- 확장 동의어 관련 키워드를 추가함
- 분해 큰 질문을 작은 질문으로 쪼갬
- 다중 쿼리 서로 다른 관점 쿼리 여러 개로 검색함

요즘 폰 배터리 왜 이렇게 빨리 닳지?

백그라운드 앱 과다 실행 배터리 사용량 확인 방법

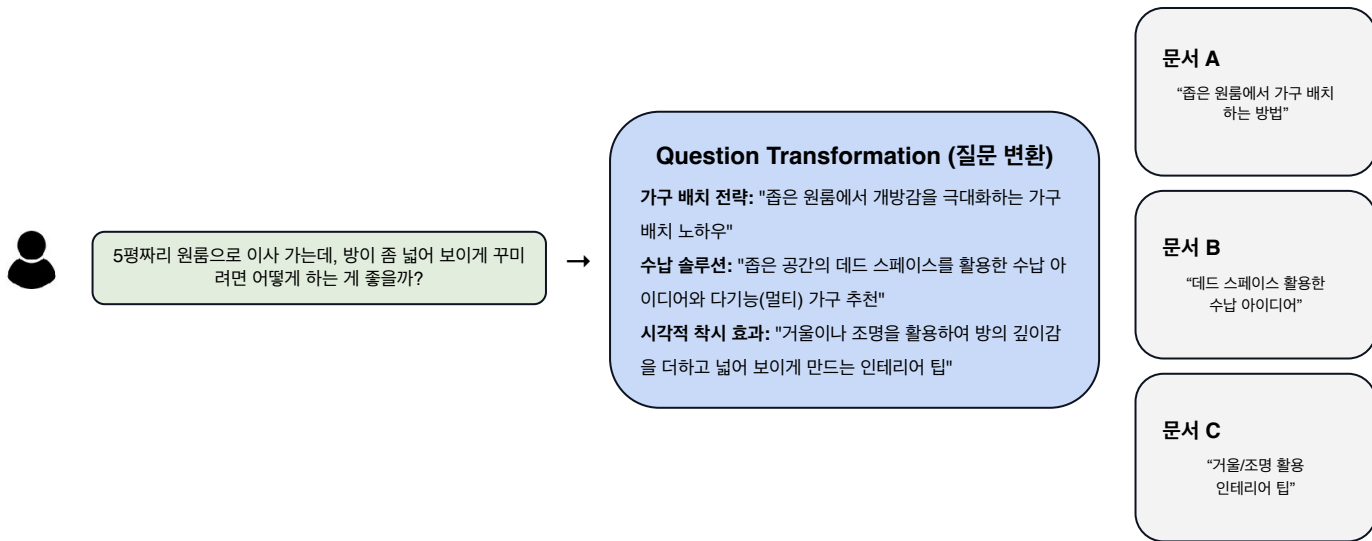
배터리 급감 원인 최근 업데이트 이후 발생 여부

배터리 성능 저하 판단 기준 교체 필요 신호

화면 밝기 5G 위치 서비스가 배터리 에 미치는 영향

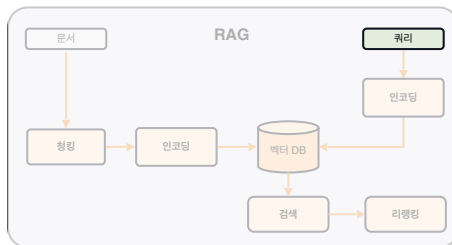
Query Transformation Intuition

- 사용자의 질문이 짧고 추상적인 경우 문서 검색이 어려움
- 질문 변환을 통해 사용자의 모호한 의도를 다각도로 구체화하여 검색 결과의 정확도를 높일 수 있음



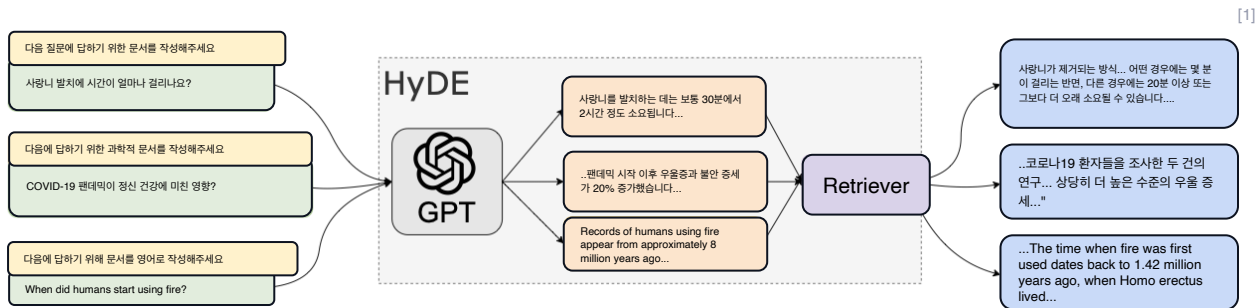
HyDE: Hypothetical Document Embedding

- Query Transformation의 대표 방법론
- 질문을 바로 임베딩하지 않고, 그 질문에 대한 가상의 답변 문서를 먼저 만든 다음 그 문서를 임베딩해서 검색하는 방식
- 사용자 질문이 짧거나 애매하면 임베딩 단서가 부족해서 검색이 흔들림
- 가상의 답변은 자연스럽게 관련 키워드와 맥락을 포함하므로 검색용 표현이 풍부해짐

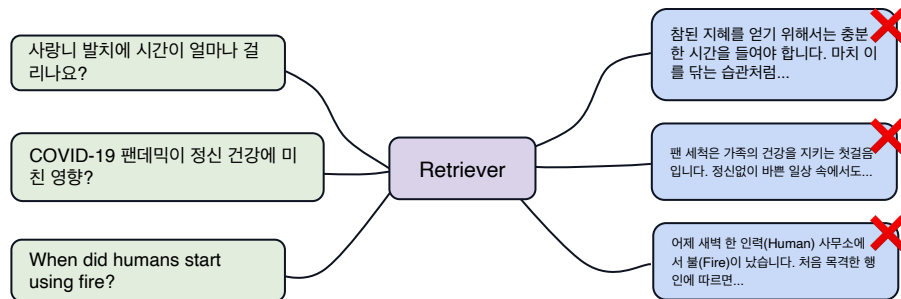


HyDE: Step-by-Step

- **Step 1: Generative Task**
 - 사용자의 쿼리를 바탕으로 LLM이 가상의 답변 문서(d')를 생성 (Zero-shot Prompting 활용)
- **Step 2: Discriminative Task**
 - 생성된 가상 문서(d')를 임베딩 모델을 통해 벡터화 (DPR 등의 모델)
 - 이 벡터를 사용하여 실제 DB 내에서 가장 유사한 **진짜 문서들**을 검색(Retrieval)



HyDE (Gao et al, 2022)



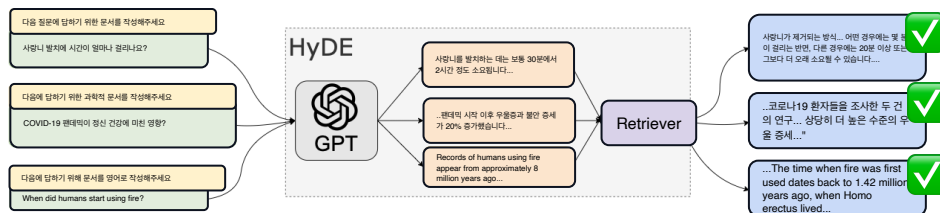
- 일반 Retriever:

질문(의문문)과 문서(평서문) 사이의 '형태적 불일치(Query-Document Gap)' 때문에, 질문에 포함된 단어만으로 검색해야 하는 한계가 있음

- HyDE:

질문을 먼저 답변 형태로 변환하여 '문서 대 문서

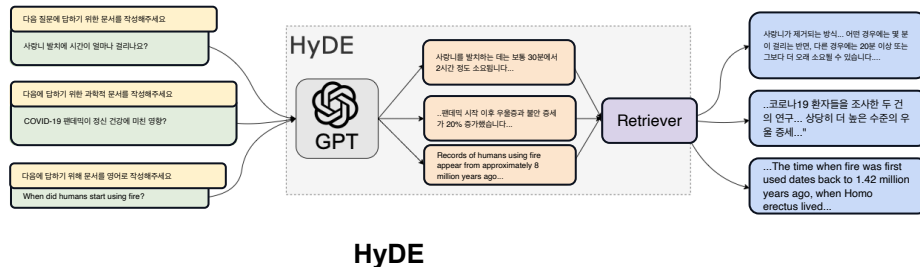
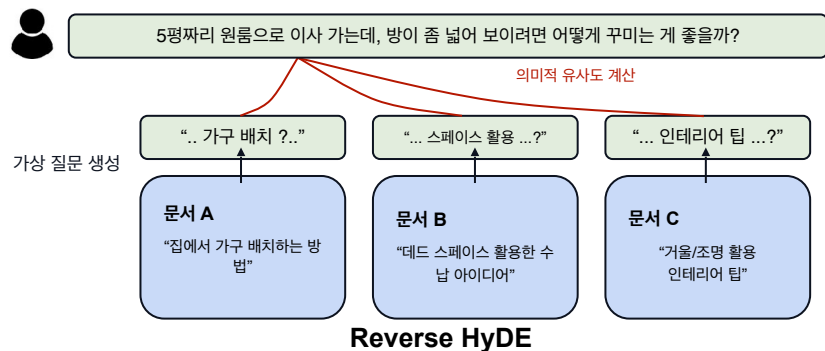
(Document-to-Document)' 매칭으로 바꿈으로써, 질문만으로는 알 수 없었던 풍부한 맥락 정보를 검색에 활용함



Reverse HyDE

방법론

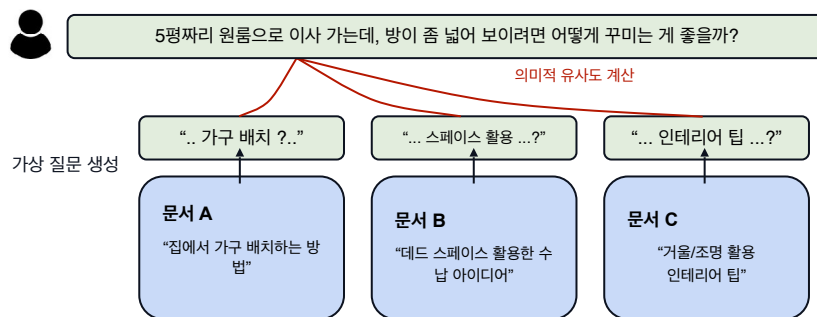
- **HyDE**: Query → 가상 Document 생성
- **Reverse HyDE**: Document → 가상 Query 생성 (메타데이터로 활용)
- 문서(Document)를 기반으로 "이 문서가 답이 될 수 있는 가상의 질문들"을 역으로 생성하여, 검색의 매개체로 활용하는 기법



Reverse HyDE

방법론

- Document → 가상의 Query 생성 → 실제 Query와의 유사도(similarity) 계산 및 비교 → 실제 질문과 생성된 가상 질문 간의 거리를 계산하여, 해당 문서가 사용자의 의도에 얼마나 부합하는지 정밀하게 측정함
- HyDE는 사용자가 질문한 시점에 가상 문서를 생성해야 해서 속도가 느리지만, Reverse HyDE는 데이터를 저장할 때 미리 질문을 뽑아둘 수 있어 실제 검색 속도가 매우 빠르다는 장점이 있음



Intuition

- 목표: 질문 하나을 가장 잘 맞추면서도 효율적인 경로를 고르는 것
 - **상황:** 여러 개의 RAG 시스템 또는 여러 개의 LLM이 있음 (ChatGPT 여러 모델 들, Gemini,)
 - 모든 질문을 가장 크고 비싼 GPT-5 같은 최신 모델에 보내지 않고 RAGRouter를 통해 저렴한 비용으로 높은 정확도 달성하자
 - 쉬운 연산은 가벼운 모델이, 복잡한 연산은 비싼 모델 활용
- 핵심 직관:
 - 활용하려는 모델이 현재 질문에 대해 원래 아는 정도 파악
 - 검색으로 주어진 문서를 얼마나 잘 활용하는 능력이 있는지 파악
 - 지금 검색된 문서 자체의 성격 파악
- 위 세 가지를 함께 보면, 어떤 모델이 정답을 낼지 더 잘 예측할 수 있음.

Query Routing

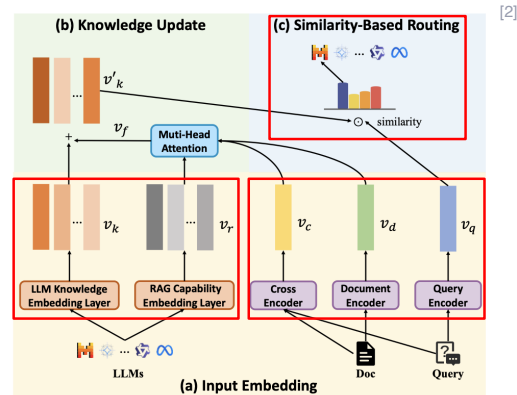
- 질문을 처리할 “경로”를 결정하는 과정
 - 예) Model Selection: 질문의 길이, 질문 유형 (QA / reasoning / summarization)에 따라 그에 맞는 모델을 선택하는 과정 (수학은 추론모델, QA는 비교적 단순 모델)
 - 한계: 모델 내부의 정적인 지식 표현에만 의존함
- RAG 상황에서는 같은 모델이라도 어떤 문서를 가져왔는지에 따라 답변 품질이 완전히 달라짐
 - 즉, 문서를 고려하지 않는 라우팅은 RAG에서 본질적으로 한계가 있음

RAG에서의 Query Routing

- Query를 어떤 RAG 시스템(또는 설정)으로 처리할지 결정하는 것
 - 예를 들어 RAG 시스템 A는 Retrieval은 강하지만 reasoning이 약하고, RAG 시스템 B는 Retrieval은 약하지만 reasoning은 강함
 - 같은 Query라도 A가 잘 푸는 경우 / B가 잘 푸는 경우가 다름
- RAG에서의 Query Routing: 단순히 '모델'이 아니라 어떤 RAG 파이프라인으로 이 질문을 처리할 것인지의 문제
- RAGRouter는 “질의 + 문서 + 모델의 RAG 활용 능력” 3가지를 함께 보고 문서를 가장 잘 활용할 수 있는 RAG 시스템이 무엇인지를 학습함

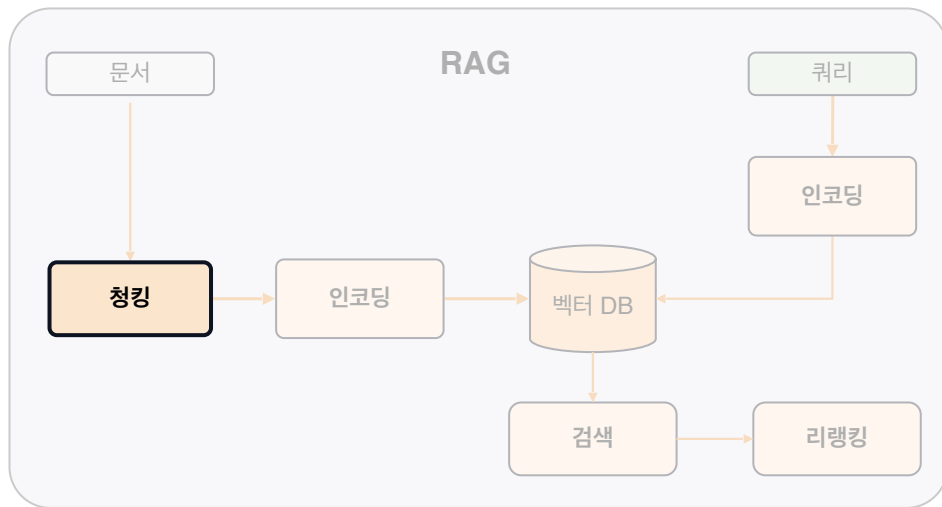
RAGRouter의 메커니즘

- 문서 임베딩 (Document Embeddings): 검색된 지식의 맥락 파악
- LLM 지식 및 RAG 능력 임베딩:
 - 특정 LLM이 해당 해당 주제에 대해 원래 알고 있는 지식의 양을 표현
 - 해당 모델이 검색된 문서를 얼마나 정확하게 활용하여 답변을 생성할 수 있는지(능력치)를 학습
- 유사도 기반 라우팅 (Similarity-Based Routing): 학습된 ‘지식/능력 벡터’와 현재의 “질의/문서 벡터” 간의 유사도를 계산하여, 가장 높은 성능을 낼 수 있는 최적의 모델 (Llama, GPT, Claude 등)로 작업을 배정(Routing)



모든 질문을 고성능 모델로 처리하던 비효율에서 벗어나, 검색된 맥락을 지능적으로 분석하여 최적의 경로를 설정하고 운영 효율을 지능화하려는 흐름

2. Chunking 전략

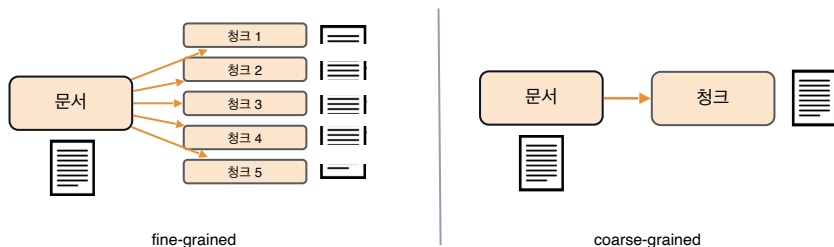


- 문서를 작은 단위인 청크로 쪼개는 것
- 작은 청크: 특정 단어나 구체적인 사실을 찾을 때 검색 정확도가 높으나, 문서 개수가 늘어나 DB에 부담 + 검색 속도가 느려짐
- 큰 청크: 답변을 생성하기 위한 풍부한 정보를 담고 있으나, 중간에 있는 정보는 잘 활용하지 못함

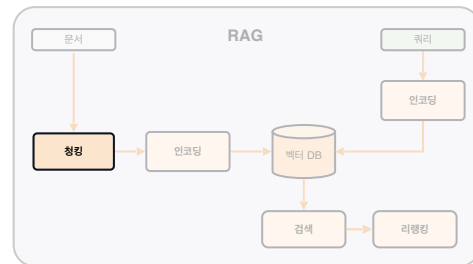
Small2Big

Fixed Chunking의 한계

- 의미적 불완전성 (semantic incompleteness)
 - 문맥을 충분히 담지 못하거나 불필요한 정보까지 함께 포함하는 문제가 있음
 - 즉, retrieval 단위와 LLM이 실제로 필요한 문맥 단위가 불일치
- Retrieval된 chunk 단위에 따른 단점
 - 작은 청크: LLM에게 전달했을 때 주변 맥락이 부족함
 - 큰 청크: 정보가 너무 섞여 있어(Noise) 벡터 검색 시 정교한 매칭이 어려움 + Lost in the middle 문제



Fixed Chunking (단순 청킹)



Small2Big

Small2Big 방식의 도입

검색은 작은(Small) 단위로, 생성은 큰(Big) 단위로 처리하는 방식 → "세부 정보를 보존하면서도 최종 답변의 질을 유지"

- Small Chunk(Child): 문서를 아주 작은 단위(예: 1~2문장)로 쪼개어 벡터 DB에 저장
검색 시 사용자의 질문과 가장 유사한 조각을 찾는 데 쓰임
- Big Chunk (Parent): 작은 조각이 포함된 더 큰 맥락(예: 문단 또는 섹션 전체)
검색된 작은 조각의 '부모' 데이터를 불러와 LLM에게 전달

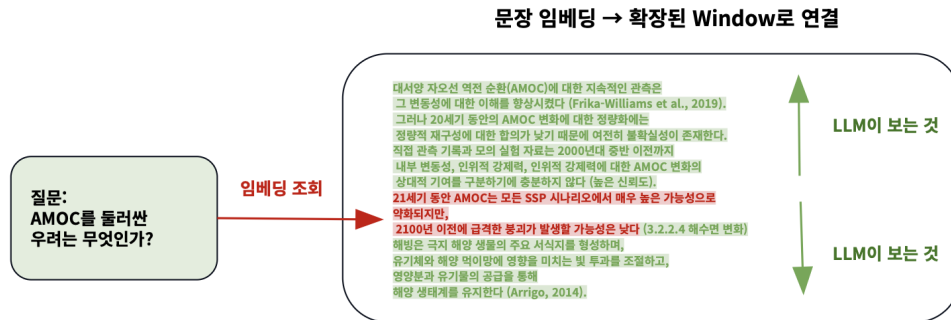
Small2Big

동작 방식

문서를 문장 단위로 임베딩한 뒤, LLM 답변 생성 단계에서 주변 문장을 포함한 더 넓은 문맥을 제공함

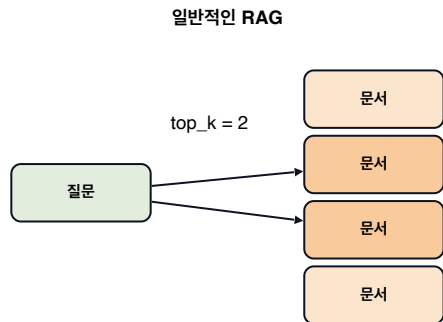
1. 문서를 **문장(sentence)** 단위로 분할
2. 각 문장을 **retrieval unit**으로 임베딩
3. 검색 시, **관련성이 높은 문장(sentence)**을 우선 검색
4. 검색된 문장을 중심으로
이전·이후 문장을 포함한 **더 큰 문맥(Big context)**을 구성
5. 확장된 문맥을 LLM 입력으로 사용하여 답변 생성

문장 단위로 검색하고, 생성 단계에서 문맥을 확장함



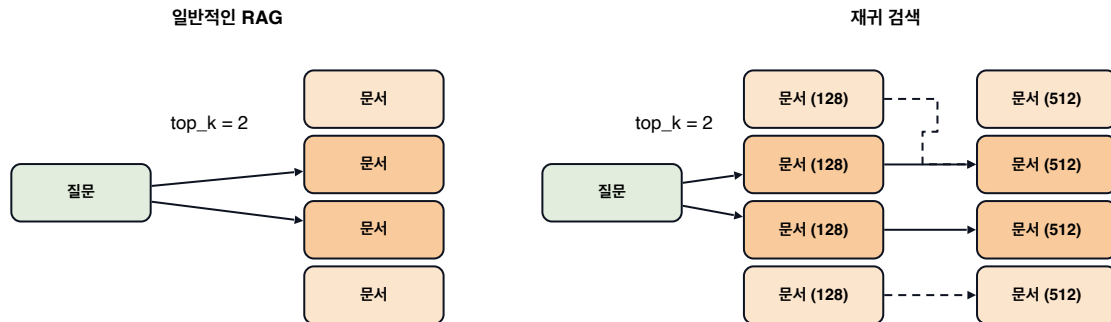
재귀적 검색

- 일반적인 RAG 방식: 질문과 가장 유사한 조각(Chunk)을 딱 한 번 찾아오는 방식
- 재귀적 검색 방식: 단순히 한 번의 검색으로 끝내지 않고, 검색된 결과가 가리키는 연관 정보나 더 깊은 단계의 데이터를 '꼬리에 꼬리를 물 듯' 계속해서 찾아가는 방식 (e.g. **Small2Big**)

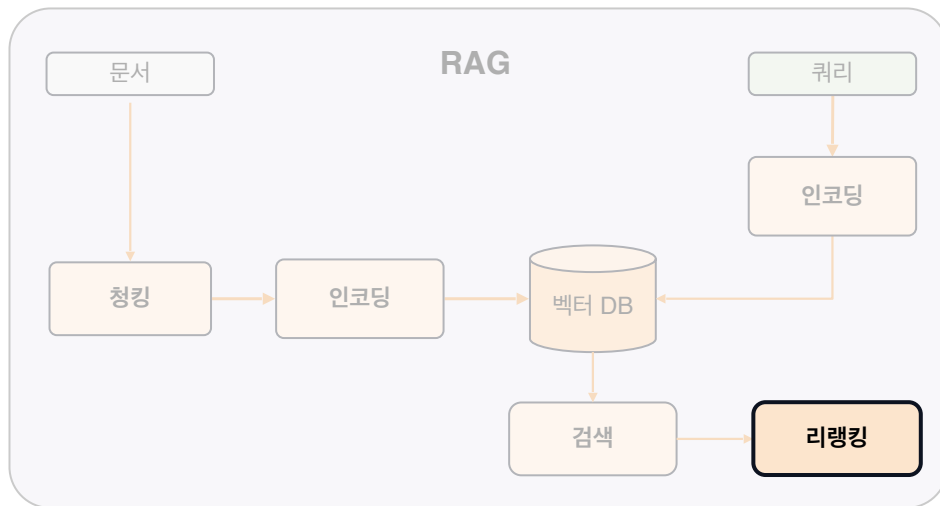


재귀적 검색

- 검색 단계
 - **1차 검색:** 질문과 관련된 '작은 조각'이나 '요약본' 검색
 - **연관 확인:** 찾은 결과가 다른 더 큰 문서나 상세 정보를 가리키는 '포인터(Link)' 역할을 하는지 확인
 - **재귀적 실행:** 만약 더 깊은 정보(부모 문서, 관련 테이블 등)가 필요하다면, 그 링크를 따라가서 **자동으로 다음 단계의 검색을 수행**



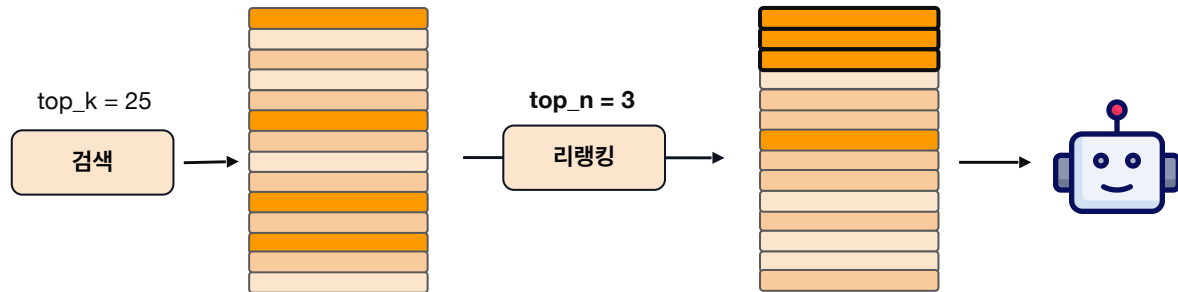
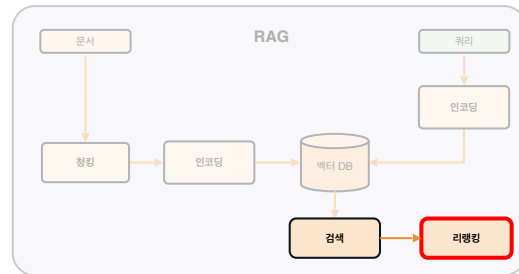
3. Reranking 전략



Recap

Reranking이란?

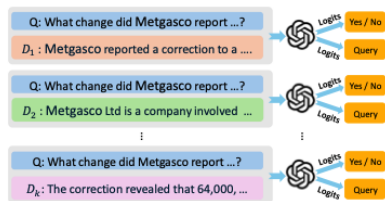
- 여러 모델 (Sparse, Dense 방식)의 검색 결과를 모두 가져와서 상위의 것들을 다시 정렬하는 방법
- 가장 중요한 정보를 **앞쪽에 배치**하여 언어 모델이 더 잘 활용하도록 도움
- “Lost in the middle” 문제 완화



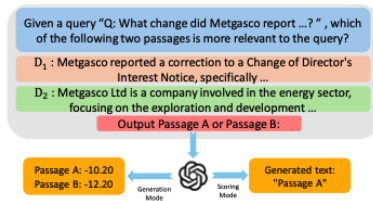
Model-based Reranking

모델 기반 리랭킹의 접근 방식

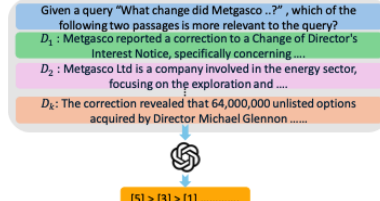
1. **Pointwise approach:** LLM이 각 문서를 하나씩 독립적으로 검토하여 관련성 점수를 매기는 방식
2. **Pairwise approach:** LLM이 두 개의 문서를 서로 비교하여 어느 쪽이 더 우수한지 판단하는 방식
3. **Listwise approach:** LLM이 전체 문서 리스트를 한꺼번에 입력받아 최적의 순서로 재배치하는 방식



(a) Pointwise approach



(b) Pairwise approach

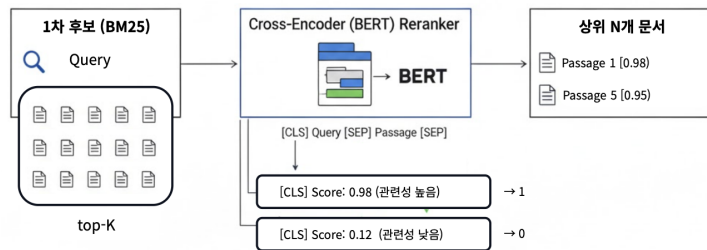


(c) Listwise approach

cf) Passage Re-Ranking with BERT (Nogueira et al, 2019)

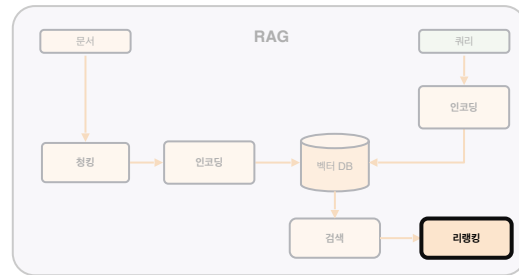
기본 Reranking

- 파이프라인: (BM25 등) 1차 후보 검색 (K개) → K개 중에서 Cross-Encoder (BERT)로 리랭킹 → 상위 문서로 답 생성
- Cross-Encoder Reranker: 질의와 문서를 함께 모델(BERT)에 입력해 relevance 점수를 예측
 - “[CLS] Query [SEP] Passage [SEP]” 형태의 입력을 사용
 - BERT의 마지막 레이어에서 [CLS] 토큰의 출력값을 활용해 해당 문서가 질문과 관련이 있는지 없는지 확률 점수를 내보냄
- 학습: BM25 (예: top-1000) 후보 내에서 positive/negative (0 또는 1)를 구성해 BERT를 학습시킨다 (fine-tuning)
- Pointwise 방법 (= 문서를 하나씩 따로 평가)



Pointwise Reranking의 한계

- 전통적인 reranker는 대부분 pointwise 방식
 - 각 query-passage 쌍을 따로 판단, 문서 간 순서 관계를 충분히 고려하지 못함
- Pointwise rerankers**
 - Query와 각 passage를 개별적으로 비교: 문서들끼리의 비교 불가능
 - 문서 간 상대적 순위 정보 부족
- Pairwise rerankers**
 - 두 문서씩 비교하지만
 - 계산 복잡도 증가 → 실용성 낮음

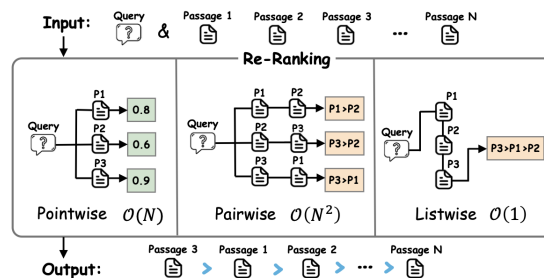


Listwise reranking이 문서 전체의 순서 & 관계를 동시에 고려할 수 있어 더 유리한 방향으로 여겨짐

Listwise-encoded Reranker

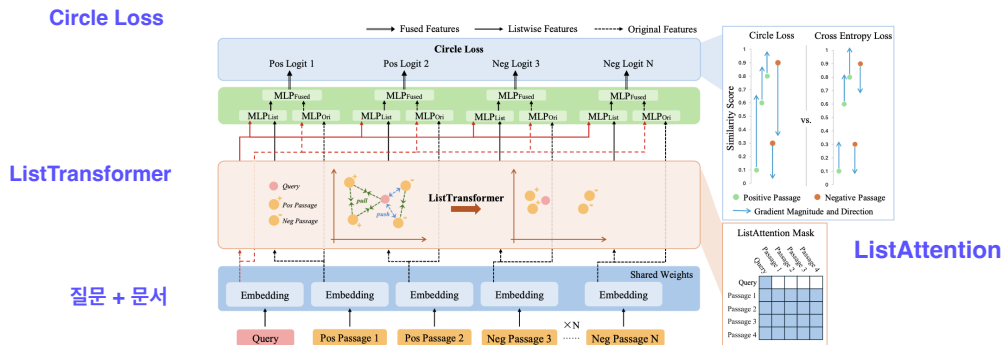
문서 전체 순위 & 관계를 고려하는 reranking 모델

- 즉, 여러 개의 후보 문서를 한 바구니에 넣고 동시에 비교하며 점수를 매기는 방식
- **Pointwise** (기존)
 - (질문, 문서1) 점수 측정 → (질문, 문서2) 점수 측정... (각개전투)
- **Listwise** (ListConRanker)
 - 질문과 함께 문서 1, 2, 3... N개를 동시에 모델에 입력하여, 각 문서가 서로에 대해 가지는 상대적인 중요도를 한꺼번에 계산
 - 결과적으로: "이 질문에는 1번 문서가 2번보다는 확실히 낮고, 3번은 아예 탄소리네!"
라는 식의 상대적 비교가 인코딩 과정에서부터 일어나기 때문에 훨씬 정교한 순위 선정이 가능해짐



구성 요소

1. 질문 하나에 대해 관련 있을 법한 문서를 검색해 모델에 모두 전달
2. ListTransformer: 추출된 여러 문서의 특징들을 한꺼번에 입력받아, 문서들 사이의 관계를 파악
 - 리스트 전체를 대상으로 global contrastive 정보를 학습: 서로 비교하며 순위를 매길 수 있도록
 - ListAttention 추가 → 문서 간 상호 비교 활성화
3. Circle Loss를 통해 정답 문서와 오답 문서를 더욱 명확하고 안정적으로 구분 지어줌



Summary

Advanced RAG (1)

- RAG 파이프라인 전반의 고도화 전략
 - Query 강화 기법
 - Chunking 전략
 - Reranking 전략



Building intelligence for the future of work